

Читаем много лишнего

Когда возникает

Как правило, возникает при желании «прикрутить еще один фильтр» к уже существующему запросу.

Как опознать

```
1  -> Seq Scan | Bitmap Heap Scan | Index [Only] Scan [Backward]
2    && 5 × rows < RRbF -- отфильтровано >80% прочитанного
3    && loops × RRbF > 100 -- и при этом больше 100 записей суммарно
```

Рекомендации

Создать [более] специализированный **индекс с WHERE-условием** или включить в индекс дополнительные поля.

Если условие фильтрации является «статичным» для ваших задач — то есть **не предполагает расширения** перечня значений в будущем — лучше использовать WHERE-индекс. В эту категорию хорошо укладываются разные boolean/enum-статусы.

Если же условие фильтрации **может принимать разные значения**, то лучше расширить индекс этими полями — как в ситуации с BitmapAnd выше.

Пример:

```
1 CREATE TABLE tbl AS
2 SELECT
3   generate_series(1, 100000) pk -- 100K "фактов"
4   , CASE
5     WHEN random() < 1::real/16 THEN NULL
6     ELSE (random() * 100)::integer -- 100 разных внешних ключей
7   END fk_own
8   , (random() < 1::real/50) critical; -- 1:50, что заявка "критичная"
9
10 CREATE INDEX ON tbl(pk);
11 CREATE INDEX ON tbl(fk_own, pk);
12
13 SELECT
14   *
15 FROM
16   tbl
17 WHERE
18   critical
19 ORDER BY
20   pk
21 LIMIT 20;
```

#	node, ms	tree, ms	rows	ratio	RRbF	node	sh.ht
	0.154	20	960			итоговые результаты	10
0	0.004	0.154	20			Limit (actual time=0.033..0.154 rows=20 loops=1) Buffers: shared hit=10	
1	0.150	0.150	20		960	-> Index Scan using tbl_pk_idx on tbl (actual time=0.032..0.150 rows=20 loops=1) Filter: critical Rows Removed by Filter: 960 Buffers: shared hit=10	10

Исправляем:

- 1 CREATE INDEX ON tbl(pk)
- 2 WHERE critical; -- добавили "статичное" условие фильтрации

#	node, ms	tree, ms	rows	ratio	node	sh.ht
	0.033	20	ИТОГОВЫЕ РЕЗУЛЬТАТЫ			8
0	0.003	0.033	20	Limit (actual time=0.020..0.033 rows=20 loops=1) Buffers: shared hit=8		
1	0.030	0.030	20	-> Index Scan using tbl_pk_idx1 on tbl (actual time=0.019..0.030 rows=20 loops=1) Buffers: shared hit=8		

Как видим, фильтрация из плана полностью ушла, а запрос стал в 5 раз быстрее.